

FraudGuard Documentation

FraudGuard — Intelligent Fraud Detection Platform

A production-quality fraud detection demo built with Streamlit, LangGraph, Ollama, and Langfuse. Combines a real-time alert queue, a conversational multi-agent investigation pipeline, and an automated invoice fraud scanner — all running fully locally with a self-hosted LLM and self-hosted observability infrastructure.

Table of Contents

1. [What Is FraudGuard](#)
 - 1.1 [Purpose](#)
 - 1.2 [Business Problem Solved](#)
 - 1.3 [Technology Stack](#)
 - 1.4 [Key Features](#)
2. [System Architecture](#)
3. [Data Layer](#)
4. [The 3-Stage Pipeline Pattern](#)
5. [Velocity Fraud Detection](#)
6. [Hallucination Reduction System](#)
7. [Invoice Fraud Detection](#)
8. [Risk Scoring](#)
9. [Alert Queue & Decisions Store](#)
10. [UI Architecture](#)
11. [Observability \(Langfuse\)](#)
12. [LangGraph Implementation Details](#)

13. [Agent Tools Reference](#)
 14. [Environment Variables](#)
 15. [Setup & Running](#)
 16. [Project Structure](#)
 17. [Dependency Decisions](#)
 18. [Known Limitations & Next Steps](#)
 19. [Notes & Operational Caveats](#)
 20. [Demo Variations](#)
-

1. What Is FraudGuard

FraudGuard is a demo-quality, locally-hosted fraud detection platform built to show what a real AI-powered fraud investigation tool looks like end-to-end. It is not a toy chat interface. Every component is wired together the way production systems are:

- Fraud signals are computed deterministically by code, not inferred by an LLM
- The LLM is used only for reasoning over verified facts — never for data collection
- Every agent invocation is traced with latency, input, and output to a dedicated observability system
- Analyst decisions are persisted to a database with attribution and timestamp
- The UI reflects real analyst workflows: ranked queues, one-click decisions, drill-down investigation, and conversational follow-up

The platform covers two fraud domains:

Transaction velocity fraud: Card-present and online account takeover patterns characterized by rapid transaction bursts, geographic impossibilities, and dormant account reactivation.

Invoice fraud: Accounts payable fraud patterns including exact duplicates, near-duplicate billing, invoice splitting to avoid approval thresholds, and ghost/fictitious vendor submissions.

1.1 Purpose

The platform enables fraud analysts to:

- Review and act on ranked alert queues in real time
- Deep-dive into individual accounts: velocity metrics, geo flags, dormancy signals, and full transaction timelines
- Converse with an AI agent to investigate suspicious patterns

- Persist analyst decisions (block / clear / escalate / monitor) to a local SQLite database
- Trace every agent run for latency, token usage, and session replay via Langfuse

1.2 Business Problem Solved

Manual fraud triage is slow, inconsistent, and error-prone. FraudGuard addresses this by:

- **Centralising alert management** — one queue, one workspace
- **Automating velocity and geo-anomaly scoring** to surface true positives faster
- **Providing an AI agent** to accelerate hypothesis testing during investigations
- **Producing a durable audit trail** of every analyst decision

1.3 Technology Stack

Layer	Technology	Notes
UI	Streamlit	Multi-page app; entry point <code>account_lookup.py</code>
Agent Framework	LangGraph	Stateful graph-based agent orchestration
LLM	Ollama (<code>qwen2.5:14b</code>)	Local inference; no data leaves the machine
Observability	Langfuse v2 (self-hosted)	Docker Compose; traces persist in Postgres volume
Database	SQLite (<code>fraudguard.db</code>)	Local only; excluded from git
Data	Pandas + synthetic CSVs	Committed under <code>synthetictables/</code>

1.4 Key Features

Alert Queue

A ranked list of flagged accounts ordered by risk score. Analysts can take one-click actions directly from the queue:

- **Block** — immediately restricts account activity
- **Clear** — removes the flag after review

- **Escalate** — routes to senior investigation team
- **Monitor** — adds to watchlist for passive surveillance

Account Lookup

A deep-dive view for any account, surfacing:

- Velocity metrics — transaction counts per rolling 5-minute window
- Geographic anomaly flags — unusual origin countries or rapid geo-shifts
- Dormancy signals — accounts reactivating after extended inactivity
- Full transaction timeline with per-transaction fraud flags

Agent Chat

A conversational fraud investigation interface powered by a LangGraph agent. The agent reasons over account data using five specialised tools (see [Section 13](#)) and produces plain-language investigation summaries. All inference runs locally via Ollama — no data is sent to external APIs.

Multi-Agent Pipeline

A three-stage workflow designed to improve reliability and reduce hallucinations. Each agent performs a distinct task:

Stage	Role	Has Tools
Data Agent	Gathers account information, transaction history, velocity metrics, and related fraud indicators	Yes
Audit Agent	Reviews findings against verified account data and identifies inconsistencies	No
Synthesis Agent	Generates a structured investigation report	No

See [Section 4](#) for full implementation details.

Ground Truth Verification

A verification layer that validates agent findings against source data before reasoning begins. This reduces hallucinated values in the final report:

- Recomputes velocity and risk metrics directly from the database
- Identifies discrepancies between the Data Agent's claimed values and actual computed values
- Flags unverified claims for the Audit Agent to correct

See [Section 6](#) for full implementation details.

Observability

Every agent run is traced to a self-hosted Langfuse v2 instance, capturing:

- End-to-end latency per run
- Token usage (prompt and completion)
- Full session history for replay and audit

See [Section 11](#) for infrastructure and trace structure.

2. System Architecture

Component Map

```

account_lookup.py          ← Streamlit entry point – Dashboard
|
├─ pages/alert_queue.py    ← Alert Queue: ranked flagged accounts, analyst
decisions
├─ pages/2_Agent_Chat.py   ← Conversational agent chat, pipeline debug panel
├─ pages/3_Invoice_Fraud.py ← Invoice fraud findings dashboard + agent chat
|
├─ agent.py                ← Velocity fraud 3-stage pipeline
|   ├─ _direct_account_investigation() ← Deterministic data collection (no LLM)
|   ├─ _verify_data_summary()         ← Ground truth verifier (no LLM)
|   └─ _build_data_agent()            ← LangGraph agent with tools (fallback
only)
|   └─ _build_reasoning_agent()        ← Stateless LLM-only agent (Audit +
Synthesis)
|       └─ run_agent()                 ← Public entrypoint, runs full pipeline
|
├─ invoice_agent.py        ← Invoice fraud 3-stage pipeline
|   └─ _build_data_agent()        ← LangGraph agent with invoice tools

```

```

|   |— _build_reasoning_agent()      ← Stateless LLM-only agent (Audit +
Synthesis)
|   |— run_invoice_agent()           ← Public entrypoint, runs full pipeline
|
|— utils.py                          ← All pure-Python computation
|   |— compute_velocity()            ← Rolling 5-min window velocity algorithm
|   |— build_alert_queue()           ← Batch scoring of all accounts
|   |— compare_accounts()            ← Multi-account side-by-side comparison
|   |— init_db() / save_decision() / get_decisions() ← SQLite decisions store
|   |— load_invoices()               ← Invoice CSV loader
|   |— detect_exact_duplicates()      ← Invoice fraud: exact match
|   |— detect_near_duplicates()       ← Invoice fraud: amount + date proximity
|   |— detect_split_billing()         ← Invoice fraud: clustering
|   |— detect_threshold_avoidance()   ← Invoice fraud: sub-threshold invoices
|   |— detect_ghost_vendors()        ← Invoice fraud: rare vendor detection
|   |— build_invoice_risk_report()    ← Runs all 5 passes, returns consolidated
dict
|
|— components.py                     ← Shared UI: render_sidebar_nav()
|— css/style.css                     ← Global styles: dark sidebar, cards, badges, pipeline
UI

```

Request Flow — Velocity Fraud Investigation

```

User types "Investigate ACC-00009"
    |
    ▼
pages/2_Agent_Chat.py
    → appends user message to session_state.messages
    → st.rerun() to render user bubble
    → on next render: calls run_agent(messages, session_id, analyst)
    |
    ▼
agent.run_agent()
    → creates Langfuse trace
    → regex search for ACC-XXXXX in user_input
    → found → _direct_account_investigation("ACC-00009")
        |
        |— lookup_account.invoke({"account_id": "ACC-00009"})

```

```

    |— analyze_velocity.invoke({"account_id": "ACC-00009"})
    |— get_transaction_history.invoke({"account_id": "ACC-00009", "limit": 10})
    |— if risk_level == "High": get_similar_flagged_accounts.invoke(...)
    |
    ▼ data_output (structured text)
    |
    ▼
_verify_data_summary(data_output)
    → re-runs compute_velocity() from database
    → regex-parses 6 claimed values from data_output text
    → compares each against actual DB values
    → returns "VERIFIED" or "DISCREPANCIES" report
    |
    ▼
Audit Agent (LangGraph, no tools)
    → receives: data_output + ground_truth_report
    → outputs: structured audit review with confidence ratings
    |
    ▼
Synthesis Agent (LangGraph, no tools)
    → receives: original prompt + data_output + audit_review
    → outputs: final formatted Fraud Investigation Report
    |
    ▼
pages/2_Agent_Chat.py
    → appends response to session_state.messages
    → stores debug dict in session_state.last_debug
    → st.rerun() to render response bubble

```

Design Principles

LLM used only for reasoning, never for data collection. Every numeric value in a report comes from Python, not from what an LLM decides to write. The LLM's job is to interpret and synthesize pre-collected facts.

Deterministic ground truth before any LLM reasoning. The Audit Agent always sees a verified comparison between what was collected and what the database actually says. It cannot reason from false premises without that being flagged.

Two independent agent architectures, one shared pattern. Velocity fraud uses direct Python invocation for data collection (more reliable). Invoice fraud uses a LangGraph tool-calling agent (acceptable because the tool set is simpler and doesn't depend on sequential calls).

Self-hosted everything. LLM (Ollama), observability (Langfuse Docker), and state (SQLite + CSVs) run locally. No runtime external API dependencies.

3. Data Layer

Synthetic Transactions (`synthetictables/transactions.csv`)

4,336 synthetic transactions across 200 accounts. Schema:

Column	Type	Description
txn_id	string	Unique transaction ID (TXN-XXXXXX)
account_id	string	Account reference (ACC-XXXXX)
timestamp	datetime	Transaction timestamp
amount	float	Transaction amount in USD
txn_type	string	Online , POS , ATM , Transfer
merchant	string	Merchant name
city	string	City where transaction occurred
lat / lon	float	Geolocation of transaction
status	string	Approved , Declined , Blocked
is_fraud	bool	Ground truth fraud label
fraud_type	string	velocity_burst , geo_anomaly , account_takeover , etc.

Column	Type	Description
velocity_flag	bool	Pre-computed: transaction is part of a velocity burst
geo_flag	bool	Pre-computed: transaction occurred in anomalous location
notes	string	Synthetic narrative notes

Synthetic Accounts (`synthetictables/accounts.csv`)

200 synthetic customer accounts. Schema:

Column	Type	Description
account_id	string	Unique account ID (ACC-XXXXXX)
customer_name	string	Full name
email	string	Email address
phone	string	Phone number
account_type	string	Savings , Checking , Business
open_date	date	Account opening date
home_city	string	Account's registered home city
home_lat / home_lon	float	Home city geolocation
credit_limit	float	Credit limit
avg_monthly_txns	int	Baseline activity level
risk_tier	string	Standard , Enhanced , High
last_active	datetime	Last recorded activity
is_dormant	bool	True if inactive for dormant_days
dormant_days	int	Days of inactivity
kyc_verified	bool	KYC completion status
country	string	Country code

Synthetic Invoices (`synthetictables/invoices.csv`)

102 synthetic invoices with 5 embedded fraud patterns. Generated by `scripts/generate_invoices.py` . Schema:

Column	Type	Description
<code>invoice_id</code>	string	Unique invoice ID (INV-XXXXX)
<code>vendor</code>	string	Vendor name
<code>amount</code>	float	Invoice amount
<code>date</code>	date	Invoice date
<code>department</code>	string	Submitting department
<code>category</code>	string	Service category
<code>approver</code>	string	Approving manager
<code>is_duplicate</code>	bool	Duplicate flag
<code>is_split</code>	bool	Split billing flag
<code>is_ghost</code>	bool	Ghost vendor flag
<code>is_threshold</code>	bool	Threshold avoidance flag
<code>fraud_type</code>	string	<code>exact_duplicate</code> , <code>near_duplicate</code> , <code>split_billing</code> , <code>threshold_avoidance</code> , <code>ghost_vendor</code>

Fraud Patterns Embedded in Invoice Data

The generator (`scripts/generate_invoices.py`) uses `random.seed(42)` for reproducibility and embeds:

- 1. **Exact duplicate:** Metro Consulting LLC , \$4,750.00, 2024-02-14 — submitted twice
- 2. **Near-duplicate:** Apex Office Supplies , \$3,200.00 submitted 2024-03-05 and 2024-03-06 (1-day drift, same amount)
- 3. **Split billing:** Sigma Analytics , 8 invoices from \$1,100–\$1,300 over 5 days in April 2024
- 4. **Threshold avoidance:** 5 invoices from various vendors between \$9,700–\$9,999

5. **Ghost vendor:** Phantom Solutions LLC , 4 invoices between \$5,000–\$15,000 — vendor exists only in fraud records

4. The 3-Stage Pipeline Pattern

Both `agent.py` and `invoice_agent.py` implement the same pattern. The rationale for splitting into three stages instead of one:

Why Three Agents

Single-agent problem: A single LLM agent doing data collection → analysis → report generation in one pass conflates three different failure modes. Data collection errors contaminate reasoning. Reasoning errors contaminate formatting. There is no separation of concerns and no opportunity to catch mistakes between stages.

Three-stage solution:

Stage	Role	Has Tools	Failure Mode Contained
Data Agent	Collects raw facts	Yes	Tool call errors, missing data
Audit Agent	Reviews facts for consistency	No	Logical errors, hallucinated values
Synthesis Agent	Writes final report	No	Formatting errors, instruction non-compliance

Each agent passes its output forward as text. The next stage reads it as its input. This is an explicit, inspectable data pipeline — not a black-box single-agent run.

Pipeline Mechanics

All three stages use `_build_reasoning_agent(system_prompt)` , which creates a minimal single-node LangGraph:

```
def _build_reasoning_agent(system_prompt: str):
    llm = ChatOllama(model=..., temperature=0)

    def call_model(state):
        response = llm.invoke([SystemMessage(content=system_prompt)] +
                               state["messages"])
        return {"messages": [response]}

    graph = StateGraph(AgentState)
    graph.add_node("agent", call_model)
    graph.set_entry_point("agent")
    graph.add_edge("agent", END)
    return graph.compile()
```

The Data Agent uses `_build_data_agent()`, which adds a `ToolNode` and a conditional edge that loops back through the tool executor until no more tool calls are pending:

```
agent → should_continue?
    └─ has tool_calls → "tools" → "agent" (loop)
    └─ no tool_calls → END
```

`temperature=0` is used throughout for maximum output determinism.

5. Velocity Fraud Detection

Core Algorithm — `compute_velocity()` in `utils.py`

The velocity computation uses a **rolling 5-minute window count**. For each transaction timestamp `t`, it counts all transactions in the window `[t - 5min, t]`. The maximum count across all windows is `max_velocity`.

```
def txns_in_window(t):
    ws = t - timedelta(minutes=5)
    return ((acc_txns["timestamp"] >= ws) & (acc_txns["timestamp"] <= t)).sum()

counts = acc_txns["timestamp"].apply(txns_in_window)
```

```
max_velocity = int(counts.max())
peak_end_ts = acc_txns.iloc[counts.argmax()]["timestamp"]
```

The peak window timestamps mark the actual worst 5-minute burst for display in the UI and report.

Why a rolling window instead of fixed buckets: Fixed time buckets (e.g., count per hour) miss bursts that straddle bucket boundaries. A rolling window catches any 5-minute burst regardless of alignment.

Data Collection — Direct Invocation vs LangGraph

When `run_agent()` receives a query containing an account ID pattern (`ACC-XXXXX`), it bypasses the LangGraph tool-calling agent entirely and calls tools directly via `_direct_account_investigation()` :

```
acc_match = re.search(r"\bACC-\w+\b", user_input, re.IGNORECASE)
if acc_match:
    data_output = _direct_account_investigation(acc_match.group(0).upper())
else:
    data_result = _build_data_agent().invoke({"messages": lc_messages})
    data_output = _build_data_summary_from_messages(data_result["messages"])
```

Why this matters: `qwen2.5:14b` and similar local models handle single tool calls reliably but struggle with chained tool calls. When asked to “investigate `ACC-00009`”, the model may call `lookup_account` and `get_transaction_history` but skip `analyze_velocity`, or call them in the wrong order, or stop after two calls. Direct Python invocation guarantees all four tools are called in the correct order every time.

The LangGraph path (`_build_data_agent()`) is preserved for open-ended conversational queries where the agent needs to decide which tools to use based on context — e.g., “Are there coordinated attacks in the dataset?”

Velocity Agent Tools

lookup_account(account_id) Queries `accounts.csv` by account ID. Returns a JSON dict with name, type, home city, KYC status, dormancy flag, dormant_days, risk tier, and open date.

get_transaction_history(account_id, limit=20) Returns the most recent `limit` transactions from `transactions.csv`, sorted descending by timestamp. Each record includes timestamp, amount, type, city, merchant, status, velocity_flag, geo_flag, and fraud_type.

analyze_velocity(account_id) Calls `compute_velocity()` on the account’s transactions and returns `max_velocity`, threshold (5), `risk_score`, `risk_level`, `geo_flags`, `total_amount`, `total_txns`,

fraud_types, and peak_window timestamps as JSON.

get_similar_flagged_accounts(account_id) Called only when risk_level is High. Finds the time window and cities of the target account's flagged transactions, then queries for other accounts with confirmed fraud transactions in the same window. Returns up to 5 similar accounts with city overlap — a signal for coordinated attacks.

record_decision(account_id, decision, notes="") Validates the decision against {"blocked", "cleared", "escalated", "monitoring"} and calls save_decision() to persist to SQLite. Allows the agent itself to record analyst decisions from within the chat conversation.

Structured Data Summary Format

After data collection, the output is always formatted as a fixed-schema text block. This is the same format whether it came from _direct_account_investigation() or _build_data_summary_from_messages() :

ACCOUNT SUMMARY

- Account ID: ACC-00009
- Customer: Jane Smith
- Account Type: Checking
- Home City: Chicago, US
- KYC Verified: Yes
- Dormant: Yes – 45 days
- Risk Tier: High

VELOCITY ANALYSIS

- Max Velocity: 10 txns in a 5-min window
- Velocity Threshold: 5
- Risk Score: 100/100
- Risk Level: High
- Geo Anomaly Flags: 13
- Total Transactions: 40
- Total Amount: \$108,010.94
- Peak Window: 2025-01-15 14:23 – 2025-01-15 14:28
- Fraud Types Detected: velocity_burst, account_takeover

RECENT TRANSACTIONS (last 10)

- 2025-01-15 14:27 | \$3,200.00 | Online | Miami, US | Approved | VELOCITY | GEO
- ...

This fixed format is what the Ground Truth Verifier and Audit Agent expect. The regex patterns in `_verify_data_summary()` are written to parse exactly these field names.

6. Hallucination Reduction System

This is the most important architectural decision in FraudGuard. Three layers work together to prevent an LLM from writing incorrect numbers in a fraud report.

The Problem

LLMs summarizing numeric data frequently introduce small errors — rounding a number differently, misremembering a count, inverting a value. In a fraud report, a wrong risk score or geo flag count is not a cosmetic issue. An analyst acts on those numbers.

Layer 1 — Deterministic Data Collection

The Data Agent stage does not use LLM reasoning for data collection on known accounts. It calls Python functions directly. There is no LLM involved in producing the data summary.

```
# agent.py - _direct_account_investigation()
acc_raw      = lookup_account.invoke({"account_id": account_id})
velocity_raw = analyze_velocity.invoke({"account_id": account_id})
txn_raw      = get_transaction_history.invoke({"account_id": account_id, "limit":
    10})
```

The summary is then assembled in Python with explicit string formatting — not by asking an LLM to write it. The LLM has zero opportunity to introduce errors at this stage.

Layer 2 — Ground Truth Verifier (`_verify_data_summary()`)

After the data summary is produced, before the Audit Agent sees it, a deterministic verification pass runs:

1. Regex-extracts the account ID from the summary: `r"Account ID:\s*(ACC-\w+)"`
2. Queries the actual CSV data for that account
3. Re-runs `compute_velocity()` to get the authoritative values
4. Regex-parses 6 claimed values from the summary text:

Field	Regex Pattern
max_velocity	r"Max Velocity:\s*(\d+)"
risk_score	r"Risk Score:\s*(\d+)"
risk_level	r"Risk Level:\s*(High Medium Low)"
geo_flags	r"Geo Anomaly Flags:\s*(\d+)"
total_txns	r"Total Transactions:\s*(\d+)"
total_amount	r"Total Amount:\s*\\$?([\d,]+\.\d*)"

5. String-compares each claimed value to the actual computed value
6. Returns either a verification confirmation or a discrepancy report listing each mismatch

Verified output:

```
GROUND TRUTH VERIFIED for ACC-00009: max_velocity=10, risk_score=100,
risk_level=High, geo_flags=13, total_txns=40, total_amount=$108,010.94.
All values match the database.
```

Discrepancy output:

```
GROUND TRUTH DISCREPANCIES – the following values in the Data Agent summary
do not match the database. Treat these as hallucinations and use the actual values:
- geo_flags: claimed 5, actual 13 ← MISMATCH
- risk_score: claimed 80, actual 100 ← MISMATCH
```

The ground truth report is injected directly into the Audit Agent’s input:

```
audit_input = f"Data Agent output:\n\n{data_output}\n\n---\n\n{ground_truth}"
```

The Audit Agent’s system prompt instructs it: “The GROUND TRUTH values are authoritative. If the Data Agent wrote a different number, the Data Agent is wrong.”

Layer 3 — Constrained Synthesis Prompt

The Synthesis Agent prompt uses angle-bracket fill-in syntax (<account ID from ACCOUNT SUMMARY>) and enforces strict output rules:

- **No parenthetical sourcing:** The model must not write (from ACCOUNT SUMMARY) or (extracted from data) — small models tend to do this unprompted
- **No “Not available”:** If a field appears anywhere in the data, the model must use it
- **Fixed threshold:** “The velocity threshold is 5. Never use any other number.”
- **Hard stop:** “Output only the report. Nothing before it, nothing after the final line of dashes.” Small models tend to add trailing commentary
- **Direct copy instruction:** “Copy values directly from the data — do not paraphrase field names”

The combination of these three layers means: data is produced by code (Layer 1), verified by code (Layer 2), and the LLM writing the final report is constrained from inventing or misquoting values (Layer 3).

Pipeline Debug Panel

Every run stores the intermediate outputs in `st.session_state.last_debug` : - `data_agent` : the full structured text summary - `ground_truth` : the verifier’s report (rendered as `st.success` or `st.error`) - `audit_agent` : the full audit review markdown

The Pipeline Debug expander in Agent Chat shows all three stages after every run, letting analysts see exactly what each stage received and produced.

7. Invoice Fraud Detection

Detection Algorithms in `utils.py`

All five detection passes are pure Pandas operations — no LLM involved.

`detect_exact_duplicates(df)`

Groups by `[vendor, amount, date]` and returns all rows where the group count > 1.

```
key    = ["vendor", "amount", "date"]
counts = df.groupby(key)["invoice_id"].transform("count")
return df[counts > 1].copy()
```

`detect_near_duplicates(df, amount_tolerance=0.01, day_window=3)`

For each vendor group, compares all invoice pairs. Returns a pair if: - Date difference $\leq$ day_window days (default 3) - Amount difference $\leq$ amount_tolerance as a fraction of the larger amount (default 1%) - The pair is NOT an exact duplicate (different date or amount)

This catches billing errors where an invoice is resubmitted with a minor adjustment — e.g., \$3,200.00 on the 5th and \$3,200.00 on the 6th.

detect_split_billing(df)

Uses a sliding window anchor approach: for each invoice date as anchor, counts invoices from the same vendor within the next SPLIT_WINDOW_DAYS days (default 7). Flags all invoices in any window with SPLIT_MIN_COUNT or more (default 4).

Why this catches it: A vendor with a single \$10,000 service submits 8 invoices of ~\$1,200 across one week. Each individual invoice looks normal. The pattern only appears when grouped by vendor over time.

detect_threshold_avoidance(df)

Simple range filter: returns all invoices where APPROVAL_THRESHOLD - THRESHOLD_BAND $\leq$ amount < APPROVAL_THRESHOLD, i.e., \$9,500–\$9,999.99 by default.

```
lower = APPROVAL_THRESHOLD - THRESHOLD_BAND # 10000 - 500 = 9500
return df[(df["amount"] >= lower) & (df["amount"] < APPROVAL_THRESHOLD)].copy()
```

detect_ghost_vendors(df, known_vendors=None)

Returns invoices from vendors appearing fewer than 3 times total in the dataset. The known_vendors parameter allows a whitelist (unused in the current implementation but available for extension).

Why low frequency matters: Legitimate vendors typically appear across multiple invoices over time. A vendor appearing in only 1–2 invoices — especially large-amount ones — is a signal for a fictitious entity created specifically for fraudulent billing.

Invoice Agent Tools

The invoice Data Agent calls all 5 tools for every audit request. Unlike the velocity agent, there is no routing logic — the full detection suite always runs:

```
get_invoice_summary → find_duplicate_invoices → find_split_billing
                    → find_threshold_avoidance → find_ghost_vendors
```

This is acceptable because invoice audits are batch operations on the whole dataset, not targeted to a specific entity.

build_invoice_risk_report(df)

The consolidated function runs all five detection passes and deduplicates the results:

```
all_flagged = pd.concat([exact_dups, near_dups, splits, threshold, ghosts])
                .drop_duplicates("invoice_id")
```

Returns a dict with counts, records per pattern, and `total_flagged_amount` (sum of all unique flagged invoice amounts). This powers the static detection dashboard on the Invoice Fraud page.

Invoice Fraud Agent — Langfuse Tracing

The invoice agent uses the context manager style of the Langfuse v2 SDK (vs. the explicit `.trace()` / `.generation()` style used in the velocity agent):

```
with langfuse.start_as_current_observation(name="invoice-fraud-agent",
    as_type="span", ...):
    with langfuse.start_as_current_observation(name="data-agent",
        as_type="generation", ...) as gen1:
        data_result = _build_data_agent().invoke(...)
        gen1.update(output=data_output)
    with langfuse.start_as_current_observation(name="audit-agent", ...) as gen2:
        ...
    with langfuse.start_as_current_observation(name="synthesis-agent", ...) as gen3:
        ...
    langfuse.set_current_trace_io(input=user_input, output=final)
```

Both tracing styles are valid in Langfuse v2. The velocity agent uses the explicit lower-level API (more control over trace vs. generation distinction). The invoice agent uses the context manager API (cleaner nesting).

8. Risk Scoring

Formula

Computed in `utils.compute_velocity()` from three signals:

```
score = 0
score += min(max_velocity / VELOCITY_THRESHOLD * 40, 60) # max 60 pts
score += min(geo_flags * 10, 30) # max 30 pts
score += 10 if acc_txns["is_fraud"].any() else 0 # 10 pts
risk_score = min(int(score), 100)
```

Signal	Points	Cap	Notes
Velocity	$(\text{max_velocity} / 5) \times 40$	60	Scales linearly; breach at velocity=5 scores 40 pts
Geo flags	$\text{geo_flags} \times 10$	30	Each geographic anomaly adds 10 pts
Known fraud	10 if any <code>is_fraud=True</code>	10	Binary — presence of confirmed fraud in history
Total		100	Capped at 100

Risk Level Classification

```
if risk_score >= 60 or max_velocity >= 15 or geo_flags >= 3:
    risk_level = "High"
elif risk_score >= 25 or max_velocity >= VELOCITY_THRESHOLD or geo_flags > 0:
    risk_level = "Medium"
else:
    risk_level = "Low"
```

Note the **OR** conditions: an account can be classified High by a single extreme signal even if the overall score is below 60. This prevents a scenario where moderate velocity + zero geo flags + no fraud history produces a misleadingly low score on a true positive.

Score Usage

The alert queue is sorted by `risk_score` descending. All accounts scoring above 0 with `risk_level` "High" or "Medium" appear in the queue. Low-risk accounts are excluded from the queue entirely

(they do not need analyst review).

9. Alert Queue & Decisions Store

SQLite Schema (fraudguard.db)

```
CREATE TABLE IF NOT EXISTS alert_decisions (  
  account_id TEXT PRIMARY KEY,  
  decision TEXT,      -- 'blocked', 'cleared', 'escalated', 'monitoring'  
  analyst TEXT,  
  notes TEXT,  
  decided_at TEXT      -- ISO 8601 datetime  
)
```

One row per account. Repeat decisions use INSERT ... ON CONFLICT DO UPDATE (upsert), so the most recent decision overwrites the previous one.

Decision States

State	UI Color	Meaning
pending	—	No decision recorded yet
blocked	Red badge	Account access blocked
cleared	Green badge	Reviewed, no action needed
escalated	Yellow badge	Referred to senior team
monitoring	Blue badge	Watching without blocking

Decisions are recorded with analyst attribution (free-text analyst name field in the UI) and timestamp. The alert queue supports undo (DELETE from alert_decisions for that account_id).

build_alert_queue(txns, accounts)

Batch-computes velocity for every account in the transactions CSV and returns a DataFrame of all High/Medium risk accounts sorted by risk_score descending:

```

for acc_id, group in txns.groupby("account_id"):
    v = compute_velocity(group)
    if v["risk_level"] in ("High", "Medium"):
        # join account metadata, append to rows

```

This runs once on page load and is cached via `@st.cache_data` .

Multi-Account Comparison (`compare_accounts()`)

The dashboard's "Compare Accounts" tab calls `compare_accounts(account_ids, txns, accounts)` , which returns a list of velocity + account profile dicts for 2–4 selected accounts. The UI renders side-by-side metric cards and two Plotly bar charts (velocity comparison, risk score comparison) with a shared threshold line at $y=5$.

10. UI Architecture

Streamlit Multi-Page App Structure

FraudGuard uses Streamlit's native multi-page structure. `account_lookup.py` is the entry point (home page). Pages are registered in `pages/` with numeric prefixes for ordering:

```

account_lookup.py      → Dashboard (/)
pages/alert_queue.py   → /alert_queue
pages/2_Agent_Chat.py  → /2_Agent_Chat
pages/3_Invoice_Fraud.py → /3_Invoice_Fraud

```

Streamlit's auto-generated sidebar navigation is hidden and replaced by a custom component:

```

[data-testid="stSidebarNav"] { display: none !important; }

```

Shared Sidebar Component (`components.py`)

`render_sidebar_nav()` is called inside with `st.sidebar:` on every page. It renders: - FraudGuard branding (icon + tagline) - Navigation links using `st.page_link()` — one for each of the 4 pages - Live agent status dots (Velocity Agent, Invoice Agent, Audit Agent, Synthesis Agent)

Using `st.page_link()` instead of manual URL links ensures Streamlit handles routing correctly within the multi-page app.

Agent Chat — Session State Pattern

The chat page uses a two-rerun pattern to ensure the user’s message bubble renders before the agent starts processing (which can take 30–60 seconds on a local LLM):

Rerun 1: Append user message to `st.session_state.messages` , set `st.session_state._run_agent_for = prompt` , call `st.rerun()` → page re-renders with user bubble visible

Rerun 2: On the next render, detect `st.session_state._run_agent_for` is set, run the agent pipeline, append assistant response, set `_run_agent_for = None` , call `st.rerun()` → page re-renders with full conversation

Suggestion buttons use a similar `pending_input` pattern:

```
if st.button(suggestion, key="sug_" + str(i)):
    st.session_state.pending_input = suggestion
    st.rerun()
```

On the next render, `pending_input` is checked and resolved to `to_send` , which triggers the same rerun chain. This is necessary because Streamlit’s `st.chat_input` and suggestion buttons cannot both fire in the same execution cycle.

CSS Architecture (`css/style.css`)

The stylesheet is loaded on every page via:

```
with open("css/style.css") as f:
    st.markdown(f"<style>{f.read()}</style>", unsafe_allow_html=True)
```

Key CSS classes:

Class	Usage
<code>.fd-header</code>	Page header with icon + title + subtitle
<code>.feat-card</code>	Dashboard feature navigation cards

Class	Usage
<code>.metric-box / .metric-row</code>	Velocity metric grid
<code>.verdict-card / .verdict-high / .verdict-medium / .verdict-low</code>	Risk verdict block
<code>.agent-alert.danger / .agent-alert.warning</code>	Inline agent signal alerts
<code>.tl-wrap / .tl-row</code>	Transaction timeline
<code>.badge / .b-block / .b-flag / .b-ok / .b-info</code>	Status badges
<code>.agent-pipeline / .pipeline-step</code>	Pipeline progress indicator
<code>.invoice-card.flagged</code>	Invoice fraud finding rows
<code>[data-testid="stChatMessage"]</code>	Native Streamlit chat bubbles with markdown table support

Chat Message Rendering

Chat history uses `st.chat_message()` + `st.markdown()` — not custom HTML divs:

```
for msg in st.session_state.messages:
    if msg["role"] == "user":
        with st.chat_message("user"):
            st.markdown(msg["content"])
    else:
        with st.chat_message("assistant", avatar="🤖"):
            st.markdown(msg["content"])
            st.caption(f"Traced in [Langfuse]({langfuse_host}) · session: {session_id[:12]}...")
```

Why native `st.chat_message()` and not HTML divs: Markdown tables, headers, and bold text do not render when markdown is placed inside HTML `<div>` elements via `unsafe_allow_html=True`. The `\n → <br>` substitution approach (used in early versions) completely breaks table rendering. `st.chat_message()` passes content through Streamlit's own markdown renderer, which handles all CommonMark syntax correctly.

The CSS for `[data-testid="stChatMessage"]` overrides the default Streamlit bubble style to match the FraudGuard dark theme while preserving markdown rendering.

11. Observability (Langfuse)

Infrastructure

Langfuse v2 runs in Docker via `docker-compose.yml` . The compose file starts two services: -
`langfuse/langfuse:2` — the Langfuse server (API + UI) - PostgreSQL — trace storage backend

Access the Langfuse UI at `http://localhost:3000` .

SDK Version Pin

`requirements.txt` pins `langfuse>=2.0.0,<3.0.0` . This is intentional and must not be changed without also upgrading the Docker image.

Why: The Langfuse v4 SDK uses OTLP-based span export. The `langfuse/langfuse:2` Docker image does not implement OTLP endpoints. Every trace flush from a v4 SDK against a v2 server returns `404 Not Found` . The v2 SDK uses the stable REST API that the v2 server exposes.

Upgrading requires changing both `requirements.txt` (`langfuse>=4.0.0`) and `docker-compose.yml` (`langfuse/langfuse:3` or later).

Trace Structure — Velocity Agent

Each `run_agent()` call produces:

```

trace: fraud-agent-run
├─ generation: data-agent
│   input:  user query
│   output: structured text summary
├─ span: ground-truth-verifier
│   input:  data summary
│   output: VERIFIED or DISCREPANCIES report
├─ generation: audit-agent
│   input:  data summary + ground truth
│   output: audit review markdown
└─ generation: synthesis-agent
    input:  original query + data summary + audit review
    output: final Fraud Investigation Report

```

Each trace carries `session_id` (UUID, generated per conversation in Streamlit session state) and `user_id` (analyst name from the sidebar text input).

Dashboard Integration

The FraudGuard home page fetches trace activity from Langfuse at TTL=60 seconds:

```
lf.get_traces(limit=20, name="fraud-agent-run")
```

Displays: total investigations run, average latency (ms → s), number of distinct analysts, and a table of the 10 most recent runs with extracted account ID, analyst, latency, and timestamp.

12. LangGraph Implementation Details

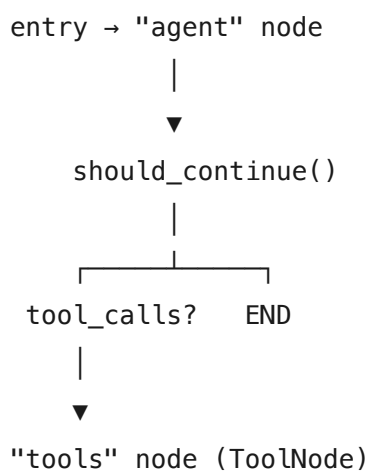
AgentState

Both agents use the same typed dict state:

```
class AgentState(TypedDict):
    messages: Annotated[list, add_messages]
```

`add_messages` is a LangGraph reducer that appends new messages to the list instead of replacing it, enabling the conversation-style accumulation needed for the tool-calling loop.

Data Agent Graph (tool-calling loop)



back to "agent"

`should_continue` checks whether the most recent message has `tool_calls` attached. If yes, the `ToolNode` executes them and appends `ToolMessage` results. The loop continues until the model produces a message with no tool calls.

Reasoning Agent Graph (single pass)

entry → "agent" node → END

No tool node, no conditional edges. The model receives the full context in its initial message and produces one response.

Tool Definition Pattern

All tools use the `@tool` decorator from `langchain_core.tools`. The docstring is the tool description passed to the model — it must be precise and directive:

```
@tool
def analyze_velocity(account_id: str) -> str:
    """
    Compute transaction velocity analysis for an account.
    Returns max transactions in any 5-minute window, geo anomaly count,
    peak window timestamps, detected fraud types, and risk level.
    Use this to assess whether velocity-based fraud is occurring.
    """
```

The docstring is what the model uses to decide which tool to call and when. Vague descriptions ("Get data for an account") cause the model to call tools in the wrong order or skip them entirely.

13. Agent Tools Reference

Velocity Fraud Tools (`agent.py`)

Tool	Inputs	Returns
lookup_account	account_id: str	Account details: name, type, home city, KYC status, dormancy flag + days, risk tier, open date
get_transaction_history	account_id: str , limit: int = 20	Recent transactions with timestamp, amount, type, city, merchant, status, velocity_flag, geo_flag, fraud_type
analyze_velocity	account_id: str	max_velocity, threshold (5), risk_score, risk_level, geo_flags, total_amount, total_txns, fraud_types, peak_window start/end
get_similar_flagged_accounts	account_id: str	Up to 5 accounts with overlapping fraud time window, with fraud_txns count and city_overlap
record_decision	account_id: str , decision: str , notes: str = ""	Confirmation string; persists to SQLite. Valid decisions: blocked / cleared / escalated / monitoring

Invoice Fraud Tools (invoice_agent.py)

Tool	Inputs	Returns
get_invoice_summary	none	total_invoices, total_amount, date_range, vendor_count, top_vendors, departments
find_duplicate_invoices	none	exact_duplicates list, near_duplicates list, total_duplicate_exposure
find_split_billing	none	split_groups by vendor (invoice_count, total_amount, date_range), total_exposure

Tool	Inputs	Returns
find_threshold_avoidance	none	invoices list, unique_vendors count, total_exposure
find_ghost_vendors	none	ghost_vendors list, vendor_names, total_exposure

14. Environment Variables

Variable	Default	Required	Description
OLLAMA_BASE_URL	http://localhost:11434	No	Ollama API endpoint. Override if running Ollama remotely.
OLLAMA_MODEL	qwen2.5:14b	No	Model name passed to ChatOllama . Must be pulled via ollama pull .
LANGFUSE_HOST	http://localhost:3000	Yes	URL of the self-hosted Langfuse server.
LANGFUSE_PUBLIC_KEY	—	Yes	Public key from Langfuse Settings → API Keys. Starts with pk-lf- .
LANGFUSE_SECRET_KEY	—	Yes	Secret key from Langfuse Settings → API Keys. Starts with sk-lf- .

All keys are loaded via `python-dotenv` at module import time. The `.env` file is gitignored. Use `.env.example` as the template.

15. Setup & Running

Prerequisites

- Python 3.11+
- [Docker Desktop](#) (for Langfuse)
- [Ollama](#) installed locally

First-Time Setup

```
# 1. Create virtual environment
python -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate
pip install -r requirements.txt

# 2. Pull the model (~9GB download, one-time)
ollama pull qwen2.5:14b

# 3. Start Langfuse
docker compose up -d
# Visit http://localhost:3000, create account, then Settings → API Keys → Create

# 4. Configure environment
cp .env.example .env
# Edit .env with your Langfuse API keys

# 5. Generate invoice data (only needed if invoices.csv is missing)
python scripts/generate_invoices.py

# 6. Launch the app
streamlit run account_lookup.py
```

Every Session

```
docker compose up -d # Langfuse (if not already running)
ollama serve         # Ollama (if not already running as a
    service)
```

```
streamlit run account_lookup.py
```

```
# App → http://localhost:8501
```

Stopping

```
docker compose down          # Stop Langfuse containers, keep trace data
docker compose down -v       # Stop and wipe Postgres volume (deletes all traces)
```

Changing the Model

The model is configurable via `.env`. Any Ollama-compatible model can be substituted:

```
OLLAMA_MODEL=llama3.1:8b
```

Note: Models below 14B tend to produce unreliable structured output in the Audit and Synthesis stages. `qwen2.5:14b` is the minimum recommended size for consistent report formatting.

16. Project Structure

```
financeagent/
|
|— account_lookup.py      # Streamlit entry point: Dashboard, account lookup,
|                          # multi-account comparison, Langfuse activity feed
|
|— agent.py               # Velocity fraud 3-stage pipeline
|                          # _direct_account_investigation() – deterministic data
collection
|                          # _verify_data_summary() – ground truth verifier
|                          # _build_data_agent() – LangGraph tool-calling agent
(fallback)
|                          # _build_reasoning_agent() – single-pass LLM (Audit +
Synthesis)
|                          # run_agent() – public endpoint
|
|— invoice_agent.py       # Invoice fraud 3-stage pipeline
|                          # _build_data_agent() – LangGraph with invoice tools
```

```

|                                     # _build_reasoning_agent() – reused pattern
|                                     # run_invoice_agent() – public entrypoint
|
|— utils.py                         # All pure-Python computation (no LLM dependencies)
|                                     # compute_velocity() – rolling 5-min window
|                                     # build_alert_queue() – batch scoring
|                                     # compare_accounts() – multi-account comparison
|                                     # init_db() / save_decision() / get_decisions() –
SQLite
|                                     # load_invoices() – CSV loader
|                                     # detect_exact_duplicates() / detect_near_duplicates()
|                                     # detect_split_billing() /
detect_threshold_avoidance()
|                                     # detect_ghost_vendors() / build_invoice_risk_report()
|
|— components.py                   # render_sidebar_nav() – shared sidebar across all
pages
|
|— requirements.txt                 # Python dependencies (langfuse pinned to <3.0.0)
|— docker-compose.yml               # Langfuse + Postgres
|— .env.example                     # Environment variable template
|— .gitignore                       # fraudguard.db, .env, __pycache__, .venv excluded
|
|— css/
|   |— style.css                    # Global UI: dark sidebar, cards, badges,
|                                     # pipeline indicator, chat message styles,
|                                     # metric boxes, verdict cards, timeline rows
|
|— pages/
|   |— alert_queue.py               # Alert Queue: ranked flagged account cards,
|   |                                     # block/clear/escalate/monitor decisions with undo,
|   |                                     # risk/status filters, analyst attribution
|   |
|   |— 2_Agent_Chat.py              # Agent Chat: suggestion prompts, conversation
history,
|   |                                     # pipeline progress indicator, pipeline debug
expander,
|   |                                     # Langfuse trace link per message
|   |
|   |— 3_Invoice_Fraud.py           # Invoice Fraud: static detection dashboard (5

```

```

patterns),
|
|           # spend by vendor donut chart, fraud pattern bar
chart,
|
|           # all invoices table with fraud highlighting,
|           # invoice agent chat tab
|
├─ synthetictables/
|   ├─ transactions.csv      # 4,336 synthetic transactions across 200 accounts
|   ├─ accounts.csv         # 200 synthetic customer accounts
|   └─ invoices.csv         # 102 invoices with 5 embedded fraud patterns
|
├─ scripts/
|   └─ generate_invoices.py  # Reproducible invoice data generator (seed=42)
|
├─ tests/
|   ├─ test_utils.py         # Velocity computation + compare_accounts tests
|   └─ test_invoice_agent.py # Invoice agent smoke tests
|
└─ docs/
    └─ superpowers/plans/
        └─ 2026-06-05-fraudguard-v2.md  # Implementation plan used during v2 build

```

17. Dependency Decisions

langfuse>=2.0.0,<3.0.0

Pinned to v2 SDK because the self-hosted Docker image is `langfuse/langfuse:2`. The v4 SDK uses OTLP export which the v2 server does not implement. Breaking this pin without updating the Docker image will silently fail to write any traces.

langchain-ollama (not langchain-openai or langchain-anthropic)

FraudGuard is fully local. No OpenAI or Anthropic API keys required. `langchain-ollama` provides the `ChatOllama` class which connects to a locally running Ollama instance over HTTP.

langgraph>=0.1.0

Used for the `StateGraph` agent loop, `ToolNode`, and `add_messages` reducer. `LangGraph` is the correct abstraction for a multi-turn tool-calling agent — not a simple `chain.invoke()`. The tool-calling loop requires conditional edges that `LangGraph` handles natively.

`langchain-core>=0.2.0`

Provides `HumanMessage`, `SystemMessage`, `AIMessage`, `BaseMessage`, `ToolMessage`, and the `@tool` decorator. Kept separate from `langchain` (full library) to minimize the dependency footprint.

`plotly>=5.0.0`

Used for the velocity comparison bar chart, risk score comparison bar chart, and vendor spend donut chart. `Plotly` integrates cleanly with `Streamlit` via `st.plotly_chart()` and supports the white-background, `Inter` font theme used across the dashboard.

`sqlite3 (stdlib)`

Analyst decisions are stored locally. No PostgreSQL, Redis, or external database needed for the decisions store. The database lives at `fraudguard.db` (`gitignored`). If the file doesn't exist, `init_db()` creates it automatically on first write.

18. Known Limitations & Next Steps

Current Limitations

Static synthetic data: The transaction and account CSVs are static. There is no streaming ingestion layer. In a real system, this would connect to a Kafka/Kinesis stream or a transaction database.

No authentication: The app has no login or role-based access. Any analyst name can be typed into the sidebar. In production this would be replaced by SSO + role enforcement.

Invoice agent lacks ground truth verification: The invoice pipeline does not have a Layer 2 verifier equivalent to the velocity agent's `_verify_data_summary()`. The invoice detection algorithms are deterministic Python, but the Audit Agent could still misquote exposure numbers. A verifier could be added by parsing the JSON tool outputs and checking the Audit Agent's claimed totals against them.

SAR draft / block action are stubs: The "Draft SAR" and "Block Account" buttons in the account lookup view show placeholder messages. These would connect to a case management API in

production.

Alert queue decisions are local only: `fraudguard.db` is not shared across instances. Multi-analyst workflows would require centralizing the decisions store.

Natural Extensions

- Replace `synthetictables/transactions.csv` with a live database query (PostgreSQL, Snowflake, BigQuery)
- Add a `generate_transactions.py` script with configurable fraud injection rates
- Extend `record_decision` tool to call an external case management API
- Add a scheduled batch job that re-runs the alert queue nightly and sends alerts for new High-risk accounts
- Add the invoice ground truth verifier (compare Audit Agent’s quoted totals against tool output JSON)
- Upgrade to Langfuse v3/v4 (requires changing both the Docker image and the SDK version)
- Add `evaluate_report()` using Langfuse’s LLM-as-judge evaluations to score report quality over time

19. Notes & Operational Caveats

Topic	Detail
<code>fraudguard.db</code>	Excluded from git via <code>.gitignore</code> . Analyst decisions are local only and not shared across environments.
Synthetic Data	All files under <code>synthetictables/</code> contain generated data only. Safe to commit and share.
Langfuse Traces	Stored in the Docker Postgres volume. Persist across <code>docker compose down</code> restarts. Wiped only by <code>docker compose down -v</code> .
Model Selection	<code>qwen2.5:14b</code> is the default. Any Ollama-compatible model can be substituted via <code>OLLAMA_MODEL</code> in <code>.env</code> .

Topic	Detail
Offline Operation	All inference, storage, and tracing is local. The platform runs without internet access after initial setup.

20. Demo Variations

Demo Type	Duration	Focus Areas
Executive	5 min	Business value, automation, alert triage speed, risk visibility
Functional	15 min	Alert queue, account lookup, agent chat, decision recording
Technical	20–30 min	Architecture, LangGraph graph, Ollama setup, Langfuse tracing, SQLite schema